Day 43: Graphs + BFS 🚀

🔥 1. Graph Basics

👉 Graph kya hota hai?

Graph ek data structure hai jisme:

Vertices (Nodes) → points

Edges → connections between nodes

Example:

1 --- 2
| |
4 --- 3

👉 Types of Graph

1. Undirected Graph

Connection dono taraf hota hai

Example: 1 — 2 (means 2 bhi 1 se connected hai)

2. Directed Graph

Ek direction hota hai

Example: 1 → 2

👉 Graph Representation

✅ 1. Adjacency Matrix

  1 2 3
1 [0 1 1]
2 [1 0 0]
3 [1 0 0]

✅ 2. Adjacency List (Most Important)

1 → 2, 3
2 → 1
3 → 1

🔥 2. BFS (Breadth First Search)

👉 BFS kya karta hai?

Graph ko level by level traverse karta hai

Queue use hoti hai

Shortest path (unweighted graph) ke liye useful

🧠 BFS Logic

Start node ko queue me daalo

Usko visited mark karo

Queue se nikaalo

Uske neighbors ko queue me daalo

Repeat until queue empty

💻 Java Code (BFS using Adjacency List)

```java
package Day_43;

import java.util.*;

class Graph {
    int V;
    ArrayList<ArrayList<Integer>> adj;

    Graph(int V) {
        this.V = V;
        adj = new ArrayList<>();

        for (int i = 0; i < V; i++) {
            adj.add(new ArrayList<>());
        }
    }

    void addEdge(int u, int v) {
        adj.get(u).add(v);
        adj.get(v).add(u); // undirected graph
    }

    void BFS(int start) {
        boolean visited[] = new boolean[V];
        Queue<Integer> q = new LinkedList<>();

        visited[start] = true;
        q.add(start);

        while (!q.isEmpty()) {
            int node = q.poll();
            System.out.print(node + " ");

            for (int neighbor : adj.get(node)) {
                if (!visited[neighbor]) {
                    visited[neighbor] = true;
                    q.add(neighbor);
                }
            }
        }
    }

    public static void main(String[] args) {
        Graph g = new Graph(5);

        g.addEdge(0, 1);
        g.addEdge(0, 2);
        g.addEdge(1, 3);
        g.addEdge(2, 4);

        System.out.println("BFS Traversal:");
        g.BFS(0);
    }
}
```

🔍 Dry Run

Graph:

0 → 1, 2

1 → 3

2 → 4

Step-by-step:

Step	Queue	Output
Start	[0]	
Pop 0	[1,2]	0
Pop 1	[2,3]	0 1
Pop 2	[3,4]	0 1 2
Pop 3	[4]	0 1 2 3
Pop 4	[]	0 1 2 3 4

🎯 Output

BFS Traversal:
0 1 2 3 4

⚡ Important Points (Interview Ready)

- ✓ BFS uses Queue
- ✓ Time Complexity → O(V + E)
- ✓ Used in:
- ✓ Shortest path
- ✓ Level order traversal
- ✓ Social networks
- ✓ Web crawling